哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

MPC Example

姓名 SongF

学号 0000000

时间 2026-04-04

2026 年 4 月 4 日

1 例子：一维小车位置控制

小车状态定义为：

$$\mathbf{x}_k = \begin{bmatrix} p_k \\ v_k \end{bmatrix}$$

其中：

- p_k: 位置
- v_k: 速度

控制输入：

$$u_k = a_k$$

即加速度。
输出只看位置：

$$y_k = p_k$$

目标：让小车位置逐步跟踪到 1 米。

2 推导主线

2.1 第 1 步：建立离散系统模型

$$\mathbf{x}_{k+1} = A\mathbf{x}_k + Bu_k, \quad y_k = C\mathbf{x}_k$$

2.2 第 2 步：写出预测输出堆叠形式

$$\mathbf{Y}_k = F\mathbf{x}_k + G\mathbf{U}_k$$

2.3 第 3 步：构造代价函数

$$J = (\mathbf{Y}_k - \mathbf{R}_k)^T Q(\mathbf{Y}_k - \mathbf{R}_k) + \mathbf{U}_k^T R\mathbf{U}_k$$

2.4 第 4 步：代入 $\mathbf{Y}_k = F\mathbf{x}_k + G\mathbf{U}_k$

得到关于 $\mathbf{U}_k$ 的二次优化问题。

2.5 第 5 步：加入约束

例如：

$$u_{\min} \leq u_k \leq u_{\max}$$

2.6 第 6 步：滚动优化

每次只执行第一步控制。

3 对应系统模型

我们最开始写的是：

$$\mathbf{x}_{k+1} = A\mathbf{x}_k + Bu_k, \quad y_k = C\mathbf{x}_k$$

代码里对应这部分：

```
A = [1 dt;
     0 1];

B = [0.5*dt^2;
     dt];

C = [1 0];
```

这个模型的物理意义是：

- 下一时刻位置 = 当前速度积分一下
- 下一时刻速度 = 当前速度 + 加速度 × 时间

属于最基础的离散双积分模型。

4 对应堆叠控制向量 $\mathbf{U}_k$

推导里定义：

$$\mathbf{U}_k = \begin{bmatrix} u_{k|k} \\ u_{k+1|k} \\ \vdots \\ u_{k+N_p-1|k} \end{bmatrix}$$

在代码里，quadprog 求出来的 u_{opt} 就是这个量：

```
u_opt = quadprog(H, f, [], [], [], [], lb, ub, [], options);
```

它实际上就是未来 N_p 步的控制序列。

5 对应预测输出公式 $\mathbf{Y}_k = F\mathbf{x}_k + G\mathbf{U}_k$

代码里这一步就是:

```
F = zeros(Np*ny, nx);
G = zeros(Np*ny, Np*nu);

for i = 1:Np
    F((i-1)*ny+1:i*ny, :) = C * (A^i);
end

for j = 1:i
    G((i-1)*ny+1:i*ny, (j-1)*nu+1:j*nu) = C * (A^(i-j)) * B;
end
```

这段代码严格对应公式:

$$F = \begin{bmatrix} CA \\ CA^2 \\ \vdots \\ CA^{N_p} \end{bmatrix}$$

和

$$G = \begin{bmatrix} CB & 0 & \cdots & 0 \\ CAB & CB & \cdots & 0 \\ CA^2B & CAB & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ CA^{N_p-1}B & CA^{N_p-2}B & \cdots & CB \end{bmatrix}$$

这里:

- i 表示预测第几步输出
- j 表示第几个未来控制量

6 对应目标函数

推导里我们写的是：

$$J = (\mathbf{Y}_k - \mathbf{R}_k)^T Q (\mathbf{Y}_k - \mathbf{R}_k) + \mathbf{U}_k^T R \mathbf{U}_k$$

代码里对应：

```
Rk = r_value * ones(Np*ny, 1);
```

这就是参考轨迹 $\mathbf{R}_k$。

然后：

```
H = 2 * (G' * Qy * G + Ru);
f = 2 * G' * Qy * (F * x - Rk);
```

这一步就是把

$$\mathbf{Y}_k = F\mathbf{x}_k + G\mathbf{U}_k$$

代入目标函数之后得到的标准二次型。

也就是：

$$J = (F\mathbf{x}_k + G\mathbf{U}_k - \mathbf{R}_k)^T Q (F\mathbf{x}_k + G\mathbf{U}_k - \mathbf{R}_k) + \mathbf{U}_k^T R \mathbf{U}_k$$

展开后可写成：

$$\min \frac{1}{2}\mathbf{U}_k^T H \mathbf{U}_k + f^T \mathbf{U}_k$$

其中：

$$H = 2(G^T Q G + R)$$

$$f = 2G^T Q (F\mathbf{x}_k - \mathbf{R}_k)$$

代码和公式是一一对应的。

7 对应约束

推导里我们写输入约束：

$$u_{\text{min}} \leq u_{k+i|k} \leq u_{\text{max}}$$

代码里就是：

```
lb = u_min * ones(Np*nu, 1);
ub = u_max * ones(Np*nu, 1);
```

因为 u_{opt} 里面堆叠了未来所有控制量，所以直接给每个元素都加上下界即可。

8 对应滚动优化

推导里说：求出未来控制序列后，只执行第一步控制。

代码里正对应这句：

```
u = U_opt(1:nu);
```

然后系统更新：

```
x = A * x + B * u;
```

下一时刻又重新构造优化问题，再求一次。这就是 MPC 的“滚动时域”。